

ድንች

- የጊዜ ገደብ:- 10 minutes
- አካባቢ:- እንደ GPU (≈ 16 GB VRAM)፣ ኢንተርኔት የለም
- የመፍትሔው መጠን:- `solution.ipynb` ≤ 1 MB
- ማከማቻ:- 5 GB

ተግባር

ኋለኛው የግምት ጨዋታ እንድትጫወቱ ይጠቁማል። እሱ እንደ ዳኛ ከአንድ ቋሚ የቃላት ስብስብ አንድ የተደበቀ ቃል ይመርጣል፣ እና እርስዎ ቢበዛ በ30 ዙሮች ውስጥ ማግኘት አለብዎት። በእያንዳንዱ ዙር ዳኛው ሁለት ቃላትን ያወዳድርና ከተደበቀው ቃል ጋር በትርጉም ይበልጥ የሚቀራረበውን ይገልጻል። እያንዳንዱ ጨዋታ ከቋሚው ጥንድ `lamp vs potato` ይጀምራል፣ ምክንያቱም እነሱ ኋለኛው ከሚወዳቸው ነገሮች ሁለቱ ናቸው። ከዚያ ፕሮግራም አንድ አዲስ ቃል ያቀርባል። በንጽጽሩ ያሸነፈው ቃል ይቀመጥና ከሚቀጥለው ሃሳብዎ ጋር ይወዳደራል። የተደበቀውን ቃል በትክክል ባቀረቡበት ቅጽበት ጨዋታውን ያሸንፋሉ። ማዛመዱ የፊደል ካፒታላይዜሽንን ከግምት ውስጥ አያስገባም። የሚያቀርቡት እያንዳንዱ ቃል በ `dataset/vocabulary.json` ውስጥ መሆን አለበት።

ከፕሮቶኮሉ እና ከዳታ ጭነት ጋር የተሟላ ምሳሌ በ `solution.ipynb` ውስጥ አለ። የ `PublicEmbeddingPlayer` classን መቀየር ይችላሉ። ፕሮግራም አንድ ጊዜ ይጀመርና ሁሉንም ጨዋታዎች በአንድ `run` ይጫወታል፤ ፕሮቶኮሉ በእያንዳንዱ ጨዋታ መጀመሪያ አዲስ `PublicEmbeddingPlayer` ይፈጥራል።

ዳኛው

ፕሮግራም አንድ JSON object ወደ ዳኛው ይልካል፣ ዳኛውም በአንድ JSON object ምላሽ ይሰጣል።

የተደበቀው ቃል ፕሮቶኮሉን ለማብራራት ብቻ የታየበት የተሠራ ምሳሌ፡-

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}          <- matches: game won
-> {"status": "win"}
```

ዙሮች ከ1 እስከ 30 ይጠቁማሉ።

የ `verdict` አማራጮች፣ `word1` ይበልጥ የቀረበ መሆኑን የሚያመለክተው `first`፣ `word2` ይበልጥ የቀረበ መሆኑን የሚያመለክተው `second` ወይም ሁለቱም ቃላት ከተደበቀው ቃል ጋር እኩል የቀረቡ መሆናቸውን የሚያመለክተው `same` ናቸው።

`winner_word` ለሚቀጥለው ንጽጽር የሚቀመጠው ቃል ነው። በ `same` ውሳኔ ጊዜ፣ የመጀመሪያው ቃል ይቆያል።

Dataset

በሁሉም `split` የሚጋራ፡-

- `dataset/vocabulary.json` — 1602 ልዩ የትንሽ ፊደል ቃላት። የተደበቀው ቃል ሁልጊዜ ከእነዚህ አንዱ ነው።
- `dataset/public_embeddings.npy` — float32፣ shape (1602, 2560)። ረድፍ `i` በቃላት ስብስቡ ውስጥ ካለው ቃል `i` ጋር ይዛመዳል። እነዚህ ይፋዊ `embeddings` ናቸው፤ ዳኛው የተለየ የግል `representation` ይጠቀማል።

`split`ዎቹ የተደበቁ ቃላት ስብስቦች ናቸው፡-

Split	ቃላት	መልሶች	የሚጠቀሙበት
test_public	120	✅ <code>dataset/test_public.json</code>	መፍትሔዎን ለማስኬድ እና ነጥብዎን በራስዎ ለማሰላት
test_leaderboard_a	120	የተደበቀ	ቀጥታ <code>leaderboard</code>
test_leaderboard_b	120	የተደበቀ	የመጨረሻ ደረጃ

የ `train split` የለም — ምንም ነገር መለያ ካላቸው ረድፎች ላይ `fit` አይደረግም።

የቀረቡ models

አስቀድመው የሰለጠኑ ሁለት embedding models ከተግባሩ ጋር ቀርበዋል፣ እና መጠቀም ይቻላል፡-

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

ሁለቱም ከአካባቢያዊ pathያቸው መጫን አለባቸው፤ እንደ "BAAI/bge-m3" ያለ Hugging Face hub id ማውረድን ያስነሳና አይሳካም፣ ምክንያቱም ዳኝነቱ offline ነው። እያንዳንዱ directory የoffline callን የሚያሳይ ሊሠራ የሚችል example.py ይዟል።

የሚገኙ libraries፡- numpy፣ torch፣ sentence-transformers። ኢንተርኔት የለም፣ ማውረዶች የሉም፣ ሌሎች packages የሉም።

ውጤት

ምንም። ይህ interactive ተግባር ነው፡- መፍትሔዎ የመልስ ፋይል አይጽፍም፤ ከላይ በተገለጸው መሠረት stdin/stdout ከዳኛው ጋር ይገናኛል።

መለኪያ

በዙር t የተገኘ ጨዋታ $1.0 - 0.02 \times \max(0, t - 10)$ ነጥብ ያገኛል፤ በ30 ዙሮች ውስጥ ያልተፈታ ጨዋታ 0 ነጥብ ያገኛል። ስለዚህ ዙሮች 1-10 1.00 ነጥብ ያገኛሉ፣ ዙር 20 0.80 ነጥብ ያገኛል፣ ዙር 30 0.60 ነጥብ ያገኛል።

የተግባርዎ ነጥብ የጨዋታዎቹ አማካይ ነጥብ $\times 100$ ሲሆን፣ በ0.00 እና 100.00 መካከል ነው።

የ10-minute ገደቡ መጀመርን፣ ዝግጅትን እና በtest set ውስጥ ያሉትን ሁሉንም 120 ጨዋታዎች የሚሸፍን አንድ በጀት ነው።

አንዴት ማስገባት እንደሚቻል

1. solution.ipynbን ይክፈቱ፣ PublicEmbeddingPlayerን ያርትዑ፣ እና እየሠራ መሆኑን ለማረጋገጥ ሁሉንም cells ያስኬዱ።
2. ከፈለጉ፣ በአካባቢዎ ይፈትሹት፡- python local_test.py solution.ipynb --limit 5። የአካባቢው ዳኛ ይፋዊ embeddingsን ስለሚጠቀም፣ ነጥቡ መመሪያ ብቻ ነው።
3. solution.ipynbን ያስቀምጡ።
4. በJupyterLab የግራ sidebar ውስጥ ያለውን Git tab ይክፈቱ።
5. solution.ipynbን stage ያድርጉ (ከእሱ አጠገብ ያለውን + icon)።
6. የcommit መልዕክት ያስገቡ እና Commitን ጠቅ ያድርጉ።
7. push ለማድረግ ወደላይ ቀስት ያለውን የደመና icon ጠቅ ያድርጉ።
8. ወደዚህ Contest ገጽ ይመለሱ እና የcommit መልዕክቱ ካቀረቡት ጋር እንዲዛመድ በማድረግ Submitን ጠቅ ያድርጉ።

ማንኛውንም አስፈላጊ ዝግጅት እና inference የሚሸፍን፣ solution.ipynb የተሰየመ በትክክል አንድ ፋይል ያስገቡ።